

“PRODUCT SEARCH AND RECOMMENDATION SYSTEM FOR SHOPPING”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

TEAM 4

CHERUKUMUDI SRI DHRUTHI : 2520030490

KACHANA MAHIMA : 2520030593

DEEPIKAA : 2520030137

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

DECLARATION

The Project Report entitled “Product Search and Recommendation System for Shopping” is a record of bonafide work of CHERUKUMUDI SRI DHRUTHI – 2520030490, KACHANA MAHIMA – 2520030593, and DEEPIKAA – 2520030137, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The work presented in this report describes the design, implementation, testing, and documentation of a Java-based product search and recommendation application using Data Structures and Algorithms.

The results embodied in this report are based on the submitted project implementation and the project data and test evidence available with the team.

CHERUKUMUDI SRI DHRUTHI – 2520030490

KACHANA MAHIMA – 2520030593

DEEPIKAA – 2520030137



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled “Product Search and Recommendation System for Shopping” is a bonafide work done and submitted by CHERUKUMUDI SRI DHRUTHI – 2520030490, KACHANA MAHIMA – 2520030593, and DEEPIKAA – 2520030137 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide

(Dr. V. Sireesha)

Signature of the HOD



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for her valuable guidance, suggestions, and continuous support throughout the development of the Product Search and Recommendation System for Shopping.

We also express our sincere thanks to Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the necessary academic support and facilities for completing this project.

We are grateful to the Department of Computer Science and Engineering for providing an environment that enabled us to apply Data Structures and Algorithms to a practical product-search and recommendation problem.

Finally, we thank all the team members for their cooperation and contribution in algorithm implementation, dataset preparation, Java Swing interface development, testing, debugging, documentation, and presentation.

CHERUKUMUDI SRI DHRUTHI – 2520030490

KACHANA MAHIMA – 2520030593

DEEPIKAA – 2520030137

Abstract

The Product Search and Recommendation System for Shopping is a Java-based Data Structures and Algorithms application developed to search, compare, filter, and recommend products from a structured shopping catalogue. The current dataset contains 5,000 product records stored in data/products.csv. The catalogue covers 14 product categories, 142 product types, and 590 brands, with product prices ranging from Rs 39 to Rs 251,999.

The application combines product browsing, requirement-based description search, fuzzy search, similarity calculation, budget-aware recommendation, aspect matching, randomized recommendations, similar-product search, cart management, session handling, and checkout. The user-facing application is implemented using Java Swing, while the core DSA modules remain separated from the graphical interface.

The project integrates multiple DSA techniques. CO1 is addressed through problem classification and algorithm-family selection. CO2 is addressed through Naive matching, KMP, Z-function, Rabin-Karp, suffix-array/LCP and related string-similarity operations. CO3 uses Wagner-Fischer edit distance, Damerau distance, Needleman-Wunsch, Smith-Waterman and optimization routines. CO4 uses Edmonds-Karp, min-cut and bipartite matching for requirement-aspect assignment. CO5 includes knapsack optimization and a 2-approximation vertex-cover routine. CO6 includes Miller-Rabin, randomized quicksort, reservoir sampling, parallel prefix sum/reduction, and universal hashing.

A major user-oriented feature is explainable recommendation. For a selected result, the system can show “WHY THIS PRODUCT?” with matching keywords/features, description similarity, category/type compatibility, and budget compatibility. The project therefore demonstrates how advanced DSA techniques can be integrated into a practical shopping application rather than being presented as isolated algorithm demonstrations.

Table Of Contents

CONTENTS	PAGE
CHAPTER 1 - INTRODUCTION	8
1.1 Overview	8
1.2 Problem Statement	9
1.3 Scope of the Project	9
1.4 Objectives	9
CHAPTER 2 - SYSTEM REQUIREMENTS	10
2.1 Functional Requirements	10
2.2 Non-Functional Requirements	11
2.3 Hardware and Software Requirements	12
2.4 Dataset Requirements	12
CHAPTER 3 - SYSTEM DESIGN	14
3.1 High-Level Architecture	14
3.2 System Flowchart	14
3.3 Data and Component Description	15
3.4 DSA and CO Mapping	16
3.5 Dataset Design	17
CHAPTER 4 - IMPLEMENTATION	18
4.1 Module-Wise Implementation	18
4.2 PSEUDOCODE	19
4.3 JAVA SWING INTERFACE	20
CHAPTER 5 - RESULTS	21
5.1 Testing Strategy	21
5.2 Functional Results	21
5.3 GUI Results	25
CHAPTER 6 — CHALLENGES & LIMITATIONS	26
6.1 Issues Faced During Development	26
6.2 Solutions Applied	26
6.3 Limitations	27
CHAPTER 7 — CONCLUSION & FUTURE WORK	28
7.2 Future Work	28
7.3 Skills Learned	29
REFERENCES	30

List of Figures

Figure No.	Caption
Fig. 1.1	Overview of the Product Search and Recommendation System
Fig. 3.1	System Architecture of the Product Search and Recommendation System
Fig. 3.2	System Flowchart
Fig. 3.3	DSA and CO Mapping
Fig. 3.4	Product Dataset Record Structure
Fig. 4.1	Java Swing Shopping Dashboard
Fig. 5.1	Product Browsing and Category Selection
Fig. 5.2	Requirement-Based Product Search
Fig. 5.3	Product Details and Explainable Recommendation
Fig. 5.4	Budget-Based Product Recommendation using Knapsack
Fig. 5.5	Aspect-Based Product Matching
Fig. 5.6	Randomized Product Recommendation
Fig. 5.7	Similar Product Recommendation
Fig. 5.8	Shopping Cart and Checkout

CHAPTER 1 - INTRODUCTION

1.1 Overview

Product Search and Recommendation System for Shopping is a Data Structures and Algorithms project that applies different algorithmic techniques to searching, retrieving, comparing, matching, and recommending products from a large shopping catalogue. The project combines string matching, fuzzy matching, similarity calculation, graph-based matching, optimization, randomization, and parallel processing within a single application.

The application works with a CSV-based product corpus stored in data/products.csv. The current dataset contains 5,000 product records. Each record is loaded into a Product object and made available to the repository, search engine, recommendation modules, and graphical interface. The repository also builds a keyword index so that search operations can work with prepared product information rather than repeatedly reparsing the file.

The user-facing application is provided through a Java Swing graphical interface. The GUI presents a shopping-oriented workflow with Login/Register, category selection, Browse, Search, Recommendations, Cart, and Checkout. The internal DSA implementations remain behind the interface so that users interact with product-search functions instead of directly selecting algorithms.

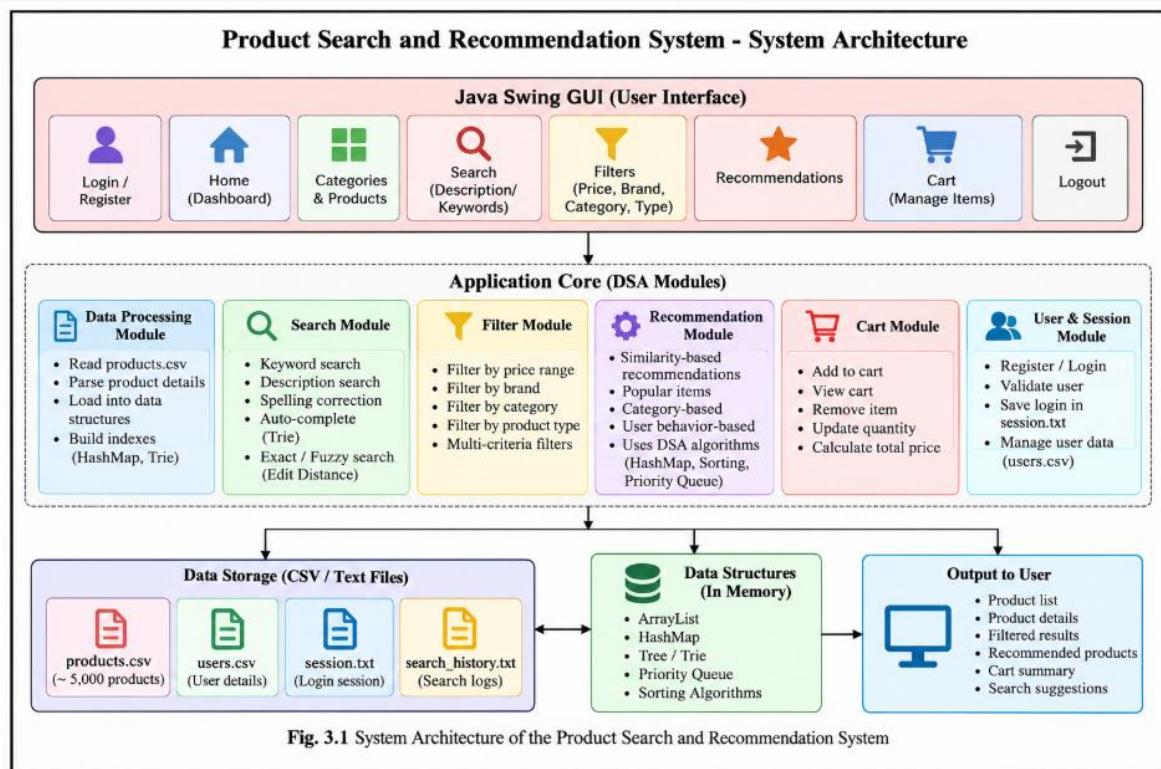


Fig. 1.1 Overview of the Product Search and Recommendation System

1.2 Problem Statement

Large shopping catalogues contain thousands of products with overlapping names, types, brands, features, and price ranges. A user may not know the exact product name and may instead describe a requirement such as “lightweight laptop with long battery life for students under 45000”. Exact string comparison is restrictive because the wording in the query may differ from the wording in the product description.

The project therefore addresses the problem of building a product-search and recommendation application that can process a large product dataset, accept natural-language-like requirements, search descriptions and product attributes, tolerate approximate wording, apply budget constraints, compare products, and explain why a recommended product is suitable.

1.3 Scope of the Project

- Loading and parsing a 5,000-record product CSV dataset.
- Supporting 14 shopping categories and 142 product types.
- Providing Login/Register and persistent session handling.
- Browsing products by category and displaying product details.
- Searching product descriptions using multiple string algorithms.
- Supporting fuzzy search and “Did you mean” correction.
- Applying minimum and maximum price filters.
- Providing budget bundles, aspect matching, surprise picks, and similar-product recommendations.
- Displaying an explainable “WHY THIS PRODUCT?” section.
- Managing a persistent user cart and checkout/order record.
- Providing a Java Swing graphical interface while keeping the core DSA implementation separate.

1.4 Objectives

The main objectives of the Product Search and Recommendation System for Shopping project are as follows:

1. To develop a Java-based product-search and recommendation application capable of processing a large product dataset and providing efficient product retrieval functionality.
2. To implement multiple pattern-matching algorithms, including Naive Pattern Matching, KMP, Z-Function, and Rabin-Karp, and integrate them into the product-search system.

3. To provide requirement-based product searching so that users can describe their needs using keywords, features, product types, and other requirements instead of entering exact product names.
4. To implement fuzzy searching using Edit Distance techniques to support product queries that contain spelling mistakes or approximate keywords.
5. To calculate similarity between user requirements and product descriptions so that relevant and closely matching products can be identified.
6. To implement dynamic programming and optimization techniques for fuzzy matching and budget-based product selection.
7. To implement graph-based matching using maximum-flow techniques to match different user requirements or product aspects with suitable products.
8. To provide explainable product recommendations by showing the matching keywords, features, description similarity, category, type, and budget suitability.
9. To implement filtering and ranking functionality based on price, category, brand, product type, and relevance.
10. To provide recommendation techniques such as budget bundles, similar products, and randomized product suggestions to improve the shopping experience.
11. To develop a Java Swing-based graphical user interface for product browsing, searching, recommendations, cart management, and checkout.
12. To implement shopping cart and checkout functionality for adding, removing, viewing, and purchasing selected products.
13. To demonstrate randomized and parallel algorithmic techniques as part of the product-search and recommendation system.
14. To evaluate the system using different product-search and recommendation scenarios and verify the correctness of the implemented functionalities.

CHAPTER 2 - SYSTEM REQUIREMENTS

2.1 Functional Requirements

2.1.1 Product Dataset Loading

The system shall load product records from data/products.csv during initialization and make the records available to search and recommendation operations.

2.1.2 Product Record Parsing

Each product record shall provide id, name, brand, category, type, price, and description information.

2.1.3 Product Storage

Parsed products shall be stored in Java data structures and accessed through ProductRepository.

2.1.4 Category Selection

The GUI shall allow users to select 14 shopping categories or All Products.

2.1.5 Product Browsing

The Browse tab shall display product ID, name, type, and price and allow details/add-to-cart actions.

2.1.6 Requirement Search

The Search tab shall accept a textual requirement and search relevant product information, especially descriptions and searchable attributes.

2.1.7 Price Filtering

The search interface shall accept minimum and maximum price values.

2.1.8 Pattern Matching

The system shall provide Naive, KMP, Z-function, and Rabin-Karp based string operations through PatternSearch.

2.1.9 Fuzzy Search

The system shall provide edit-distance based spelling correction and suggestions.

2.1.10 Similarity Search

The system shall calculate similarity using suffix-array/LCP and sequence-alignment techniques where appropriate.

2.1.11 Requirement Explanation

The system shall explain why a product is suitable by reporting matching evidence and budget/category compatibility.

2.1.12 Budget Recommendation

The system shall support budget-aware bundle selection using the knapsack routine.

2.1.13 Aspect Matching

The system shall match requirement aspects to candidate products using bipartite matching.

2.1.14 Surprise Recommendation

The system shall provide randomized category-based product selections.

2.1.15 Similar Product Recommendation

The system shall find similar products using description similarity, type, brand, and price-related signals.

2.1.16 Cart Management

The system shall support adding products, deleting products, displaying quantities, and calculating totals.

2.1.17 Checkout

The system shall create an order record and clear the cart after successful checkout.

2.1.18 Session Handling

The system shall save and read the active user from session.txt.

2.1.19 Input Validation

The system shall handle empty queries, invalid product IDs, invalid budgets, and invalid login information without unexpected termination.

2.1.20 Java Swing Interface

The system shall provide the shopping workflow through a Java Swing GUI.

2.2 Non-Functional Requirements

2.2.1 Usability

The GUI should provide a simple shopping-oriented interface without requiring users to understand the internal algorithms.

2.2.2 Performance

The catalogue should be loaded once and reusable data structures should reduce repeated file processing.

2.2.3 Scalability

The architecture should allow the product dataset to grow without changing the overall application workflow.

2.2.4 Reliability

Invalid input and missing resources should be handled through controlled messages.

2.2.5 Maintainability

Core DSA classes and GUI code should remain modular and independently understandable.

2.2.7 Data Consistency

The CSV record structure should remain consistent so the repository can parse each product correctly.

2.2.8 Error Handling

Search, cart, authentication, and checkout operations should respond clearly to invalid conditions.

2.2.9 Extensibility

New search, recommendation, and GUI features should be addable without redesigning the complete application.

2.3 Hardware and Software Requirements

Item	Requirement
Programming Language	Java
GUI Technology	Java Swing
IDE	Visual Studio Code
Data Storage	CSV and text files
Database	Not used
External Libraries	None
Compilation	<code>javac -d bin src*.java</code>
GUI Execution	<code>java -cp bin SmartShoppingGUI</code>
Console Execution	<code>java -cp bin Main</code>

2.4 Dataset Requirements

The current data/products.csv contains 5,000 product records. The dataset covers 14 categories, 142 product types and 590 brands

Field	Description
id	Unique product identifier
name	Product name
brand	Brand
category	Shopping category
type	Product type
price	Product price in rupees
description	Detailed text used for search and recommendation evidence

CHAPTER 3 - SYSTEM DESIGN

3.1 High-Level Architecture

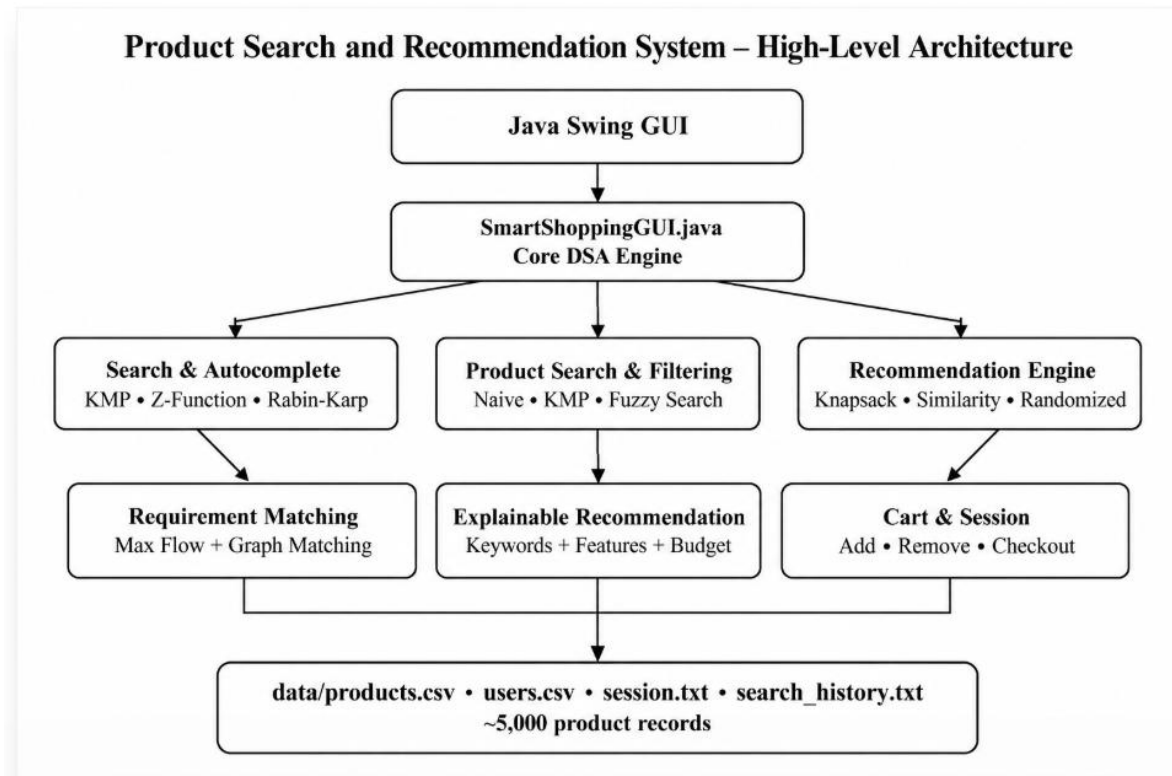


Fig. 3.1 System Architecture of the Product Search and Recommendation System

The high-level architecture of the **Product Search and Recommendation System** consists of several interconnected layers. The system starts with the **Java Swing GUI**, which provides the user interface for login, browsing products, searching, filtering, recommendations, cart management, and logout. User requests are passed to **SmartShoppingGUI.java**, which acts as the core interface connecting the GUI with the DSA-based processing modules. The system then uses different modules such as **Search and Autocomplete**, **Product Search and Filtering**, and the **Recommendation Engine**. These modules apply algorithms such as KMP, Z-Function, Rabin-Karp, fuzzy search, knapsack, similarity matching, and randomized techniques. The **Requirement Matching** module uses maximum-flow and graph matching to match user requirements with suitable products, while the **Explainable Recommendation** module identifies matching keywords, features, and budget suitability. The **Cart and Session** module manages adding, removing, and checking out products. Finally, the system stores and retrieves information from files such as products.csv, users.csv, session.txt, and search_history.txt, which contain the product catalogue and user-related data.

3.2 System Flowchart

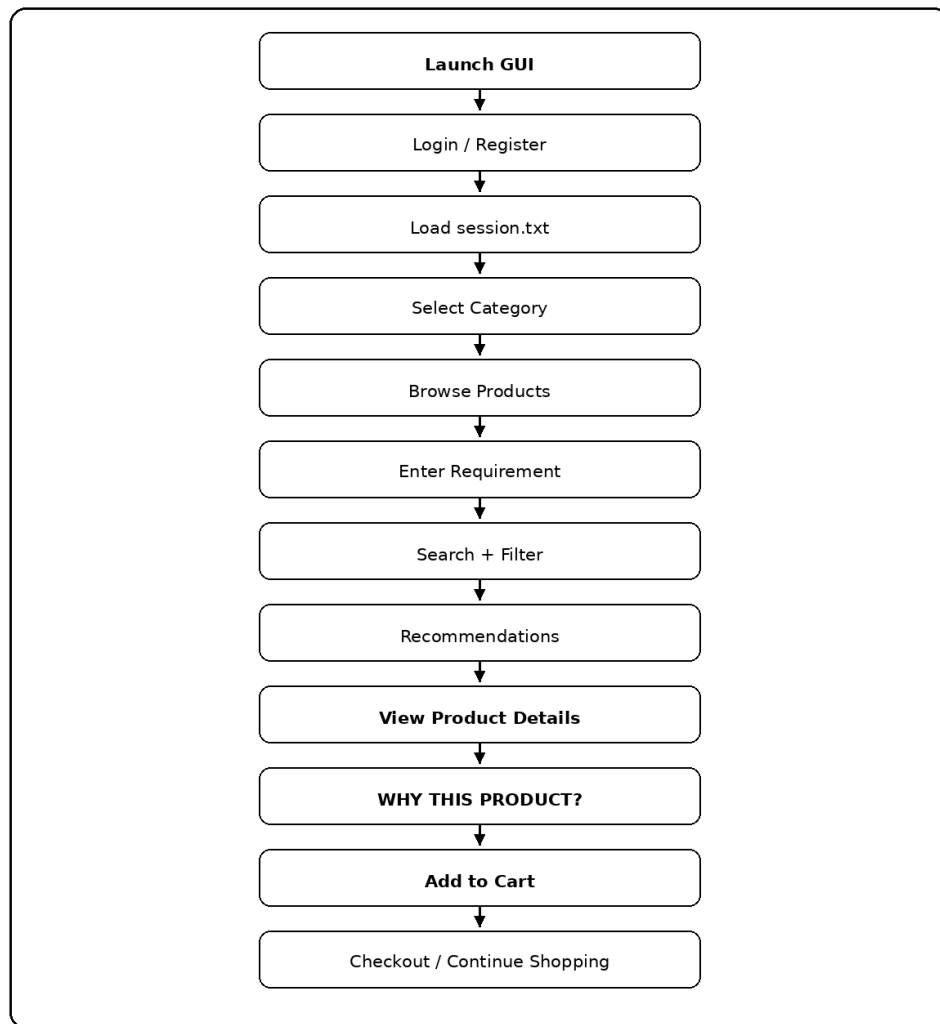


Fig. 3.2 System Flowchart

The system begins with GUI launch and session checking. A user can login/register, choose a category, browse products, enter a requirement, search/filter the catalogue, open recommendations, inspect product details, add products to the cart, and checkout or continue shopping.

3.3 Data and Component Description

Component	Main Responsibility
SmartShoppingGUI.java	Java Swing interaction layer; login, browsing, search, recommendations, cart and checkout.
ProductRepository.java	Loads products.csv, stores products and builds the keyword index.
SearchEngine.java	Combines search signals, scoring, diversification and explainability.
PatternSearch.java	Naive, KMP, Z-function and Rabin-Karp string matching.
SimilaritySearch.java	Suffix array, Kasai LCP, longest common substring, Needleman-Wunsch and Smith-Waterman.
FuzzySearch.java	Wagner-Fischer edit distance, Damerau distance and

	spelling suggestions.
NetworkFlow.java	Edmonds-Karp maximum flow, min-cut and bipartite matching.
GreedyRecommendation.java	Knapsack, Optimal BST, TSP, 2-approximate vertex cover and exact vertex cover.
ParallelSearch.java	Miller-Rabin, randomized quicksort, reservoir sampling, parallel prefix sum/reduce.
ProblemClassifier.java	Maps query characteristics to problem classes and algorithm families.
Auth.java	Registration, login, password hashing and persistent session handling.
Cart.java	Cart IDs, quantities, totals and cart-file persistence.

3.4 DSA and CO Mapping

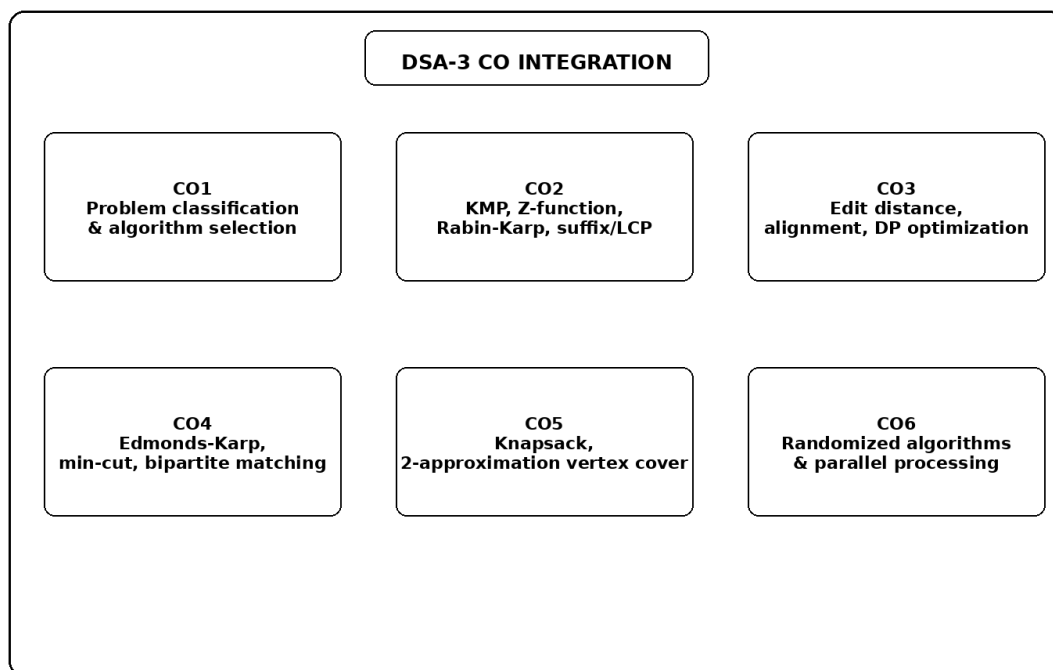


Fig. 3.3 DSA and CO Mapping

CO	Implementation	Application Use
CO1	ProblemClassifier	Identifies query/problem characteristics and suggests an algorithm family.
CO2	PatternSearch + SimilaritySearch	KMP, Z-function, Rabin-Karp, suffix array, LCP and string similarity for product search.
CO3	FuzzySearch + optimization	Edit distance, Damerau distance, sequence alignment and budget optimization.
CO4	NetworkFlow	Maximum flow, min-cut and bipartite matching for requirement-

		aspect assignment.
CO5	GreedyRecommendation	Knapsack and 2-approximation vertex cover for optimization demonstrations.
CO6	ParallelSearch	Randomization, Miller-Rabin, randomized quicksort, reservoir sampling and parallel processing.

3.5 Dataset Design

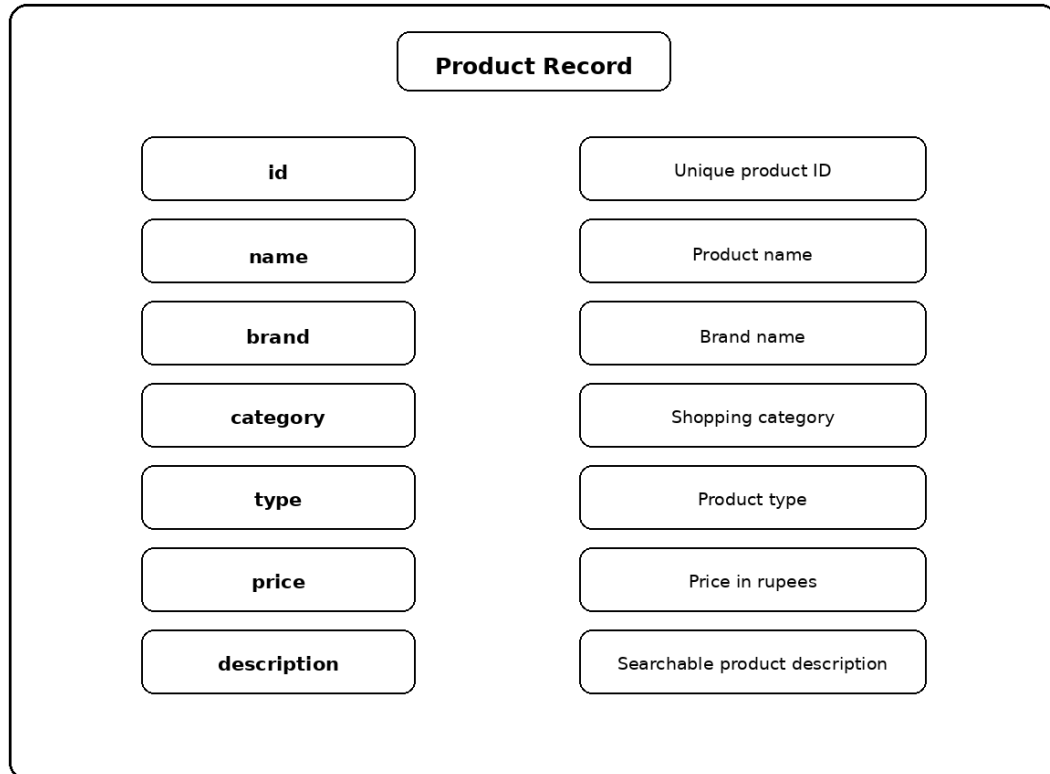


Fig. 3.4 Product Dataset Record Structure

The product record is represented by seven logical fields. The description is particularly important because requirement-based searching uses description text together with name, brand, category and type signals. The dataset is kept as a CSV file so that the complete project remains portable and editable in VS Code.

CHAPTER 4 - IMPLEMENTATION

4.1.1 Product Loading

ProductRepository.load reads products.csv, creates Product objects, records categories, and prepares the inverted keyword index.

4.1.2 Product Search

SearchEngine accepts a requirement, extracts keywords/budget, obtains candidate products and computes relevance scores.

4.1.3 Naive Pattern Matching

PatternSearch.naiveSearch performs direct pattern comparisons.

4.1.4 KMP

PatternSearch builds the LPS array and uses KMP search/count operations.

4.1.5 Z-Function

PatternSearch computes the Z array and supports prefix-oriented product-name matching.

4.1.6 Rabin-Karp

PatternSearch uses rolling hash and verification to locate patterns.

4.1.7 Suffix Array and LCP

SimilaritySearch builds suffix-array information and Kasai LCP values.

4.1.8 Sequence Alignment

Needleman-Wunsch and Smith-Waterman provide global/local alignment scores.

4.1.9 Fuzzy Search

FuzzySearch implements Wagner-Fischer edit distance and Damerau distance and can suggest corrected query wording.

4.1.10 Bipartite Matching

NetworkFlow obtains aspect-to-product matching through maximum-flow based matching.

4.1.11 Budget Optimization

GreedyRecommendation.knapsackPick selects a high-value product bundle within the requested budget.

4.1.12 Random Recommendation

ParallelSearch.reservoirSample selects a bounded set of category products.

4.1.13 Similar Product Search

The GUI combines description similarity, type/brand matches and price closeness, then ranks candidates using randomized quicksort.

4.1.14 Cart and Checkout

Cart stores product IDs and quantities, calculates totals, and checkout creates an order record.

4.1.15 Java Swing GUI

SmartShoppingGUI provides Login/Register, Browse, Search, Recommendations, Cart, category selection, product details, and checkout.

4.2 PSEUDOCODE

Product Search

SEARCH_PRODUCTS(query, category, minPrice, maxPrice)

1. Validate query.
2. Optionally request fuzzy correction.
3. Extract keywords and budget.
4. Retrieve candidates.
5. Apply category and price constraints.
6. Evaluate matching/similarity signals.
7. Compute relevance and suitability.
8. Diversify and return ranked hits.

Explain Product Suitability

EXPLAIN_PRODUCT(hit, keywords)

1. Read product.
2. Identify matching keywords/features.
3. Calculate description similarity evidence.
4. Compare category/type and budget.
5. Produce YES, PARTIALLY, or NO.
6. Display “WHY THIS PRODUCT?” evidence.

Budget Bundle

BUDGET_BUNDLE(query, budget)

1. Search relevant products inside budget.
2. Convert relevance to value and price to weight.
3. Apply 0/1 knapsack.
4. Display selected bundle, total cost and remaining budget.

Aspect Matching

ASPECT_MATCH(aspects)

1. Split requirement into aspects.
2. Search candidate products.

3. Create adjacency matrix.
4. Run bipartite matching through maximum flow.
5. Display each aspect and assigned product.

4.3 JAVA SWING INTERFACE

The Java Swing GUI is implemented in SmartShoppingGUI.java using JFrame, JPanel, CardLayout, JTabbedPane, JTable, JTextField, JTextArea, JComboBox, JButton, JScrollPane, JOptionPane and table models. The main tabs are Browse, Search, Recommendations and Cart. The Search tab supports requirement text and price filters. Recommendations contains Best Bundle, Aspect Matching, Surprise Picks and Similar Products. Cart provides add, delete, total and checkout operations.

ID	Product Name	Category	Price
1001	Samsung Smartphone Prime 100	Smartphone	10149
1002	Xiaomi Laptop Pro 107	Laptop	8959
1003	HP Headphones Edge 114	Headphones	10745
1004	Realme Smartwatch Elite 121	Smartwatch	19854
1005	OnePlus Tablet Elite 128	Tablet	13049
1006	Dell Power Bank Pro 135	Power Bank	5010
1007	Boat Bluetooth Speaker Nova 142	Bluetooth Speaker	8680
1008	Apple Monitor Neo 149	Monitor	4599
1009	Sony Keyboard Neo 156	Keyboard	4576
1010	Lenovo Mouse Pro 163	Mouse	550
1011	Samsung Camera Core 170	Camera	55248
1012	Xiaomi Earbuds Elite 177	Earbuds	14880
1013	Apple Smartphone Lite 184	Smartphone	42909
1014	Sony Laptop Plus 191	Laptop	90859
1015	Lenovo Headphones Plus 198	Headphones	11375
1016	Samsung Smartwatch Pro 205	Smartwatch	7974
1017	Xiaomi Tablet Neo 212	Tablet	3329

Fig. 4.1 Java Swing Shopping Dashboard

CHAPTER 5 - RESULTS

5.1 Testing Strategy

Test Input / Action	Expected Result	Observed Result
Launch GUI	Shopping GUI opens	GUI opens through SmartShoppingGUI
Select Electronics	Electronics products displayed	Category browsing supported
gaming laptop under 50000	Relevant products within budget	7 products shown in recorded test
Price filter	Products restricted by range	Minimum/maximum fields supported
Product details	Full product information	Details dialog available
WHY THIS PRODUCT?	Evidence-based explanation	Matching keywords, similarity, category/type and budget shown
Best bundle	Budget-aware bundle	Knapsack implementation connected
Aspect matching	Aspect-to-product assignment	Network-flow matching connected
Surprise picks	Random products	Reservoir sampling connected
Similar products	Similar results	Similarity + randomized sorting connected
Cart	Add/delete/total	Cart module connected
Checkout	Order saved and cart cleared	Checkout writes orders.txt

5.2 Functional Results

5.2.1 Product Catalogue Loading

5.2.2 Product Browsing and Category Selection

The Browse tab allows the user to select a product category and view the available products with their ID, name, type, and price. The interface also provides options to view complete product details and add a selected product to the cart.

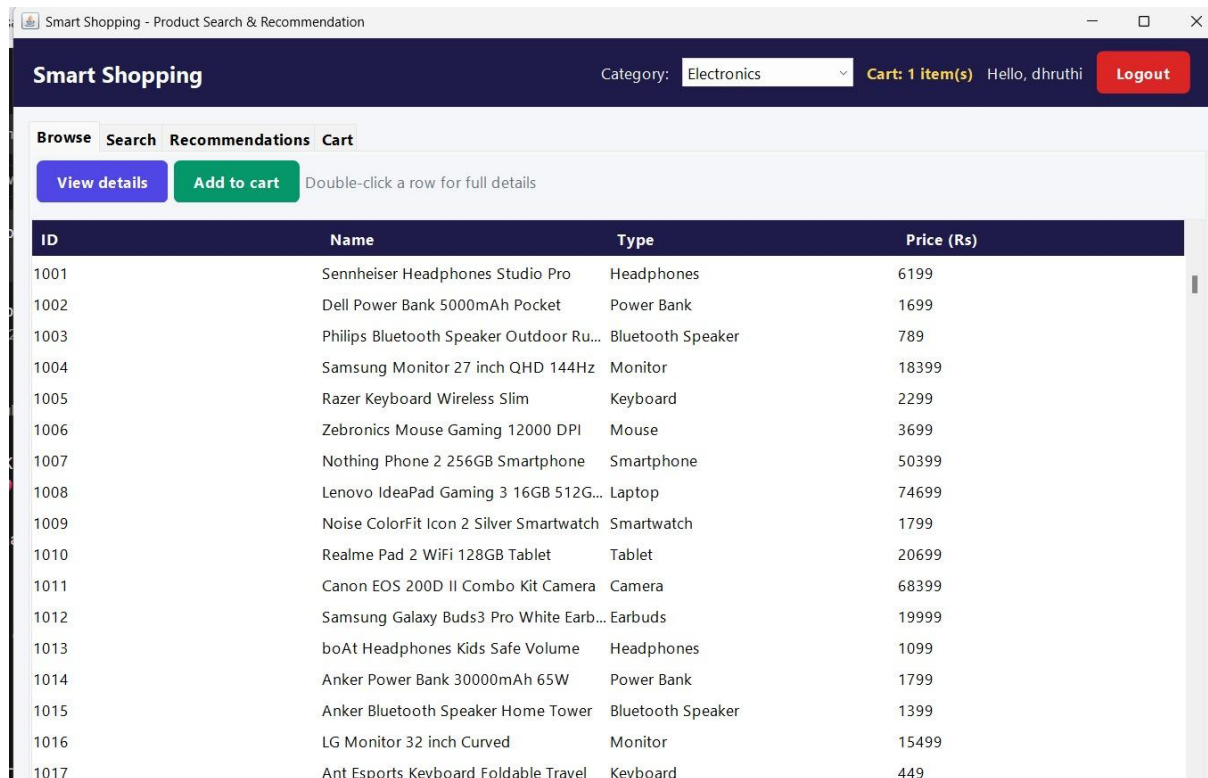


Fig. 5.1 Product Browsing and Category Selection

5.2.3 Requirement-Based Product Search

The Search tab accepts a natural-language product requirement and optional minimum and maximum price values. In the shown test, the requirement “i want a smartphone” produced relevant smartphone products ranked with a relevance score and suitability indicator.

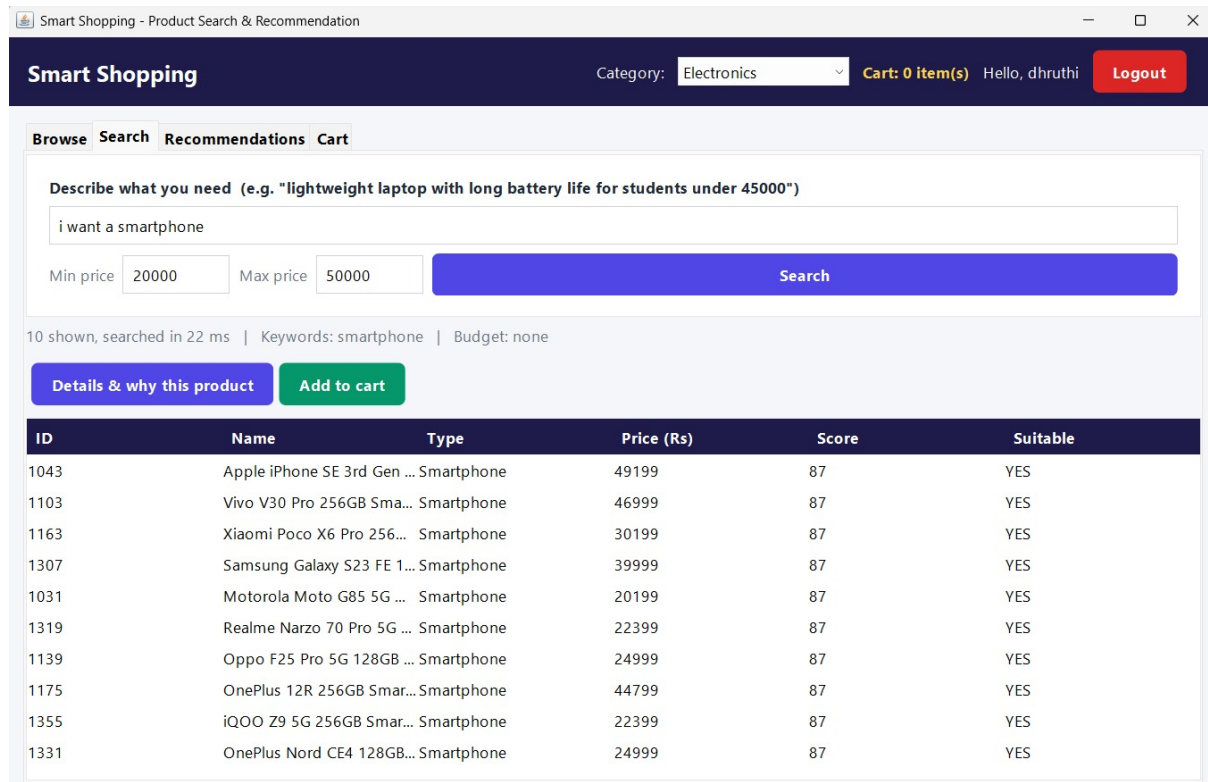


Fig. 5.2 Requirement-Based Product Search

5.2.4 Product Details and Explainable Recommendation

The product details view displays the selected product's ID, name, brand, category, type, price, and description. The “WHY THIS PRODUCT?” section provides an explanation based on matching keywords/features and description relevance, helping the user understand the recommendation.

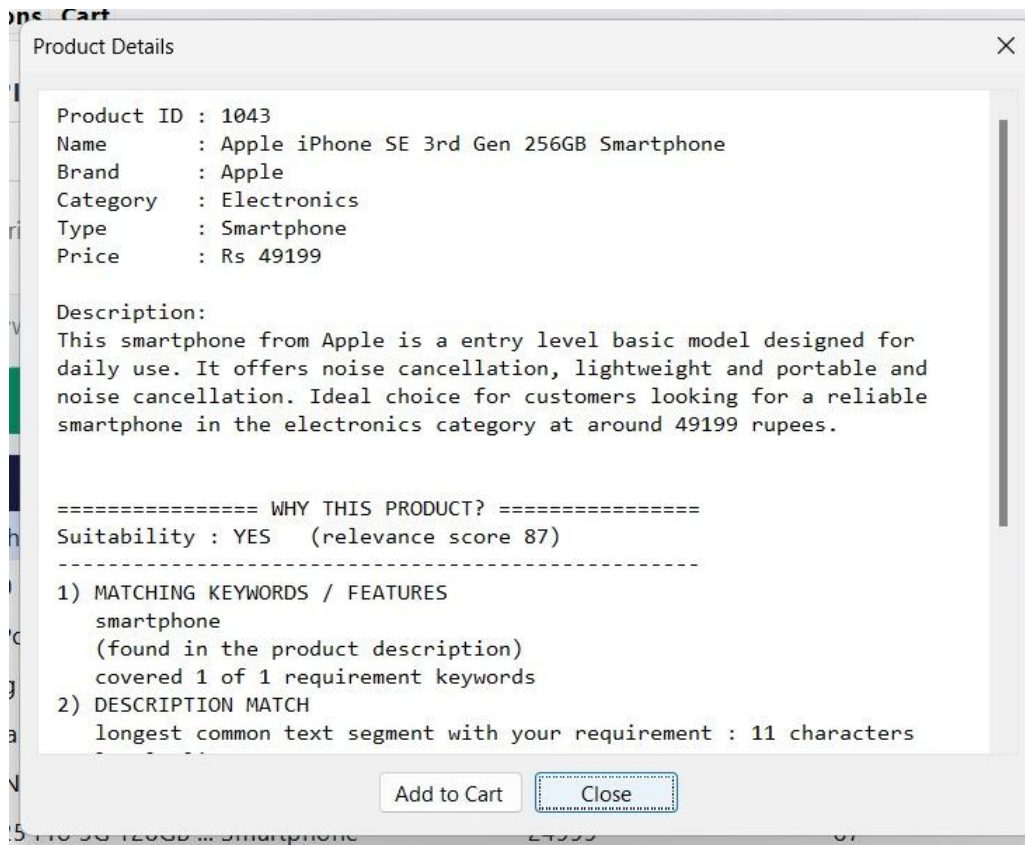


Fig. 5.3 Product Details and Explainable Recommendation

5.2.5 Budget-Based Recommendation

The Best Bundle feature applies a budget constraint and selects a combination of relevant products using the knapsack-based optimization module. The displayed result shows the selected products, total price, remaining budget, and total relevance achieved.

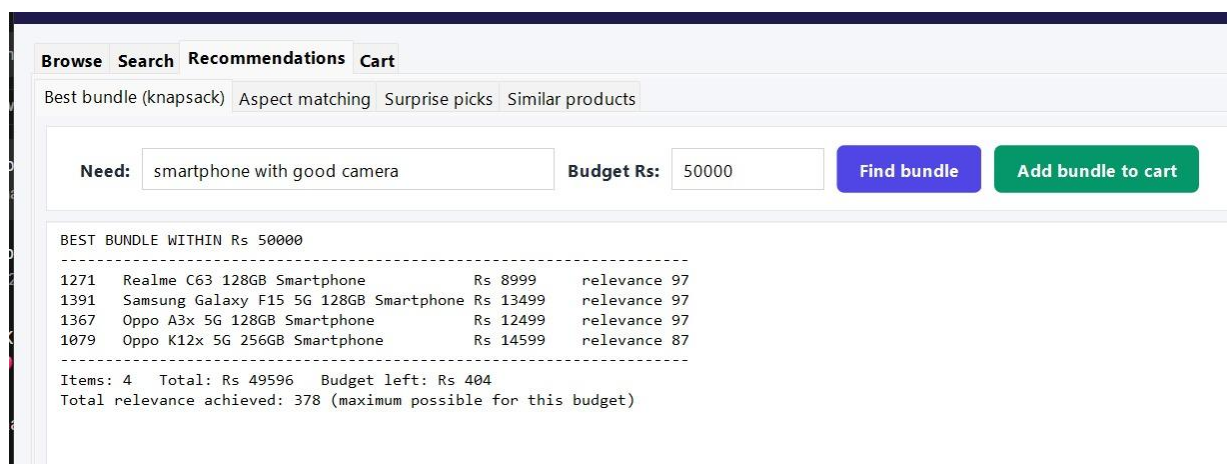


Fig. 5.4 Budget-Based Product Recommendation using Knapsack

5.2.6 Aspect-Based Product Matching

The Aspect Matching feature accepts multiple product requirements and attempts to assign each aspect to a distinct suitable product. The result reports successful assignments, unmatched aspects, and the maximum matching size obtained through the graph-based matching module.

ASPECT -> PRODUCT ASSIGNMENT

good camera	-> [1107] Sony Alpha a6400 Body Only Camera	Rs 82999
5G	-> no distinct product covers this aspect	
AMOLED display	-> no distinct product covers this aspect	
long battery life	-> [1047] Fujifilm X100V Body Only Camera	Rs 133999
fast charging	-> [1419] Sony Alpha a6400 Combo Kit Camera	Rs 99599

Maximum matching size = 3 of 5 aspects.

Fig. 5.5 Aspect-Based Product Matching

5.2.7 Randomized Product Recommendation

The Surprise Picks feature uses randomized selection to present a set of products from the selected category. This supports exploratory shopping and demonstrates the use of randomized algorithms within the application.

SURPRISE PICKS - Electronics

1389	Cosmic Byte Keyboard Silent Office	Rs 3099	Keyboard
1166	Lenovo Tab M10 Plus WiFi Cellular 128GB.	Rs 20799	Tablet
1174	Zebronics Mouse Wired USB	Rs 519	Mouse
1340	ViewSonic Monitor 27 inch 4K	Rs 17099	Monitor
1030	Dell Mouse Trackball	Rs 219	Mouse

Fig. 5.6 Randomized Product Recommendation

5.2.8 Similar Product Recommendation

The Similar Products feature takes a selected product ID and returns other products with high similarity scores. The output demonstrates similarity-based recommendation using product characteristics and text-related signals.

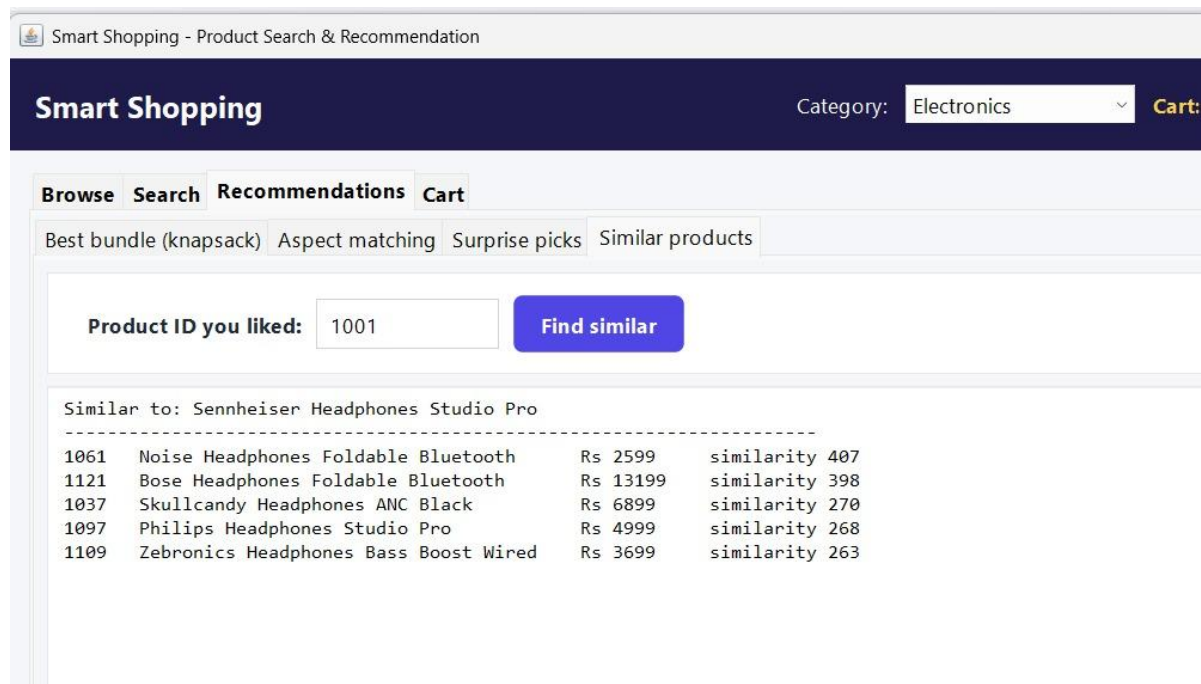


Fig. 5.7 Similar Product Recommendation

5.2.9 Shopping Cart and Checkout

The Cart tab displays selected products, quantities, unit prices, and subtotals. Users can add products, delete selected products, and proceed to checkout. This completes the shopping workflow after search and recommendation.

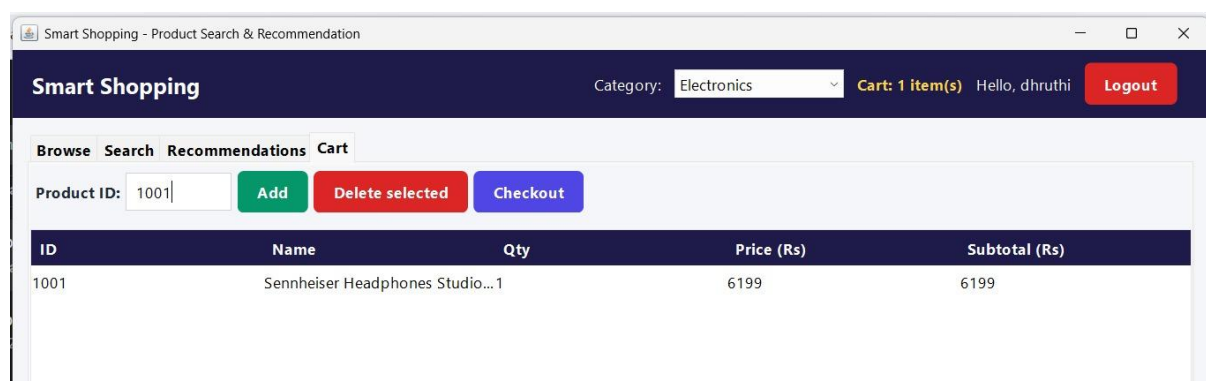


Fig. 5.8 Shopping Cart and Checkout

5.3 GUI Results

The Java Swing interface integrates the complete shopping workflow into a single desktop application., aspect matching, randomized recommendations, similarity-based recommendations, and cart management.

The current project catalogue contains 5,000 product records, 14 categories, 142 product types and 590 brands. The current price range is Rs 39 to Rs 251,999.

CHAPTER 6 — CHALLENGES & LIMITATIONS

6.1.1 Integrating Multiple Search Algorithms

Different string algorithms have different inputs and matching strategies. The project combines them through SearchEngine so the GUI remains simple.

6.1.2 Handling a Large Product Catalogue

A 5,000-record CSV requires initialization and indexing so queries do not repeatedly perform expensive file parsing.

6.1.3 Requirement Interpretation

Users describe products using multiple words, features and budgets instead of exact names.

6.1.4 Fuzzy Search

Spelling variations and wording differences require edit-distance based correction.

6.1.5 Explainable Recommendations

The system must identify the evidence behind a recommendation rather than only outputting a score.

6.1.6 Requirement-Aspect Matching

Multiple requirement aspects may need distinct products, creating a graph matching problem.

6.1.7 Budget Optimization

Selecting relevant products under a fixed budget is an optimization problem.

6.1.8 GUI Integration

Existing DSA modules had to be connected to Swing without duplicating algorithm logic.

6.2 Solutions Applied

6.2.1 Search Integration

SearchEngine combines multiple matching signals before producing ranked results.

6.2.2 Catalogue Preparation

ProductRepository loads the catalogue and constructs a keyword index.

6.2.3 Fuzzy Correction

FuzzySearch provides edit-distance and suggestion support.

6.2.4 Explainability

SearchEngine.explain reports matched keywords/features, description similarity, category/type and budget evidence.

6.2.5 Graph Matching

NetworkFlow provides bipartite matching and maximum-flow operations.

6.2.6 Optimization

GreedyRecommendation provides knapsack and additional optimization routines.

6.2.7 Randomization and Parallelism

ParallelSearch provides reservoir sampling, randomized quicksort, Miller-Rabin, prefix sum and reduction.

6.2.8 GUI Separation

SmartShoppingGUI remains a separate front-end class and calls the existing DSA/business classes.

6.3 Limitations

- Product information is stored in CSV/text files rather than a production database.
- The dataset provenance and commercial pricing accuracy are not established as live marketplace data; the catalogue is a project dataset containing real-world-style brands and categories.
- Recommendation quality depends on the text available in product descriptions and the implemented matching signals.
- Parallel processing may not improve every workload because thread-management overhead can outweigh the benefit for small inputs.
- Advanced algorithms are integrated for course demonstration and their practical contribution depends on the query type.
- The application is a Java Swing desktop application and does not provide a web/mobile deployment.
- Cart/order persistence is file-based and intended for an academic project environment.

CHAPTER 7 — CONCLUSION & FUTURE WORK

The Product Search and Recommendation System for Shopping demonstrates the practical application of Data Structures and Algorithms to searching, matching, comparing, optimizing, and recommending products from a large catalogue. The project is implemented in Java and currently operates on a 5,000-record product dataset stored in data/products.csv.

The system combines Naive Pattern Matching, KMP, Z-function, Rabin-Karp, suffix-array/LCP processing, edit distance, sequence alignment, network flow, bipartite matching, knapsack optimization, approximation, randomized algorithms and parallel processing. These techniques are connected to actual shopping functions rather than being presented as isolated demonstrations.

The Java Swing interface presents product browsing, requirement search, filters, recommendations, product details, explainability, cart management and checkout as normal user operations. The internal DSA algorithms remain separated from the GUI, allowing the interface and algorithm implementations to be maintained independently.

The explainable recommendation mechanism provides a direct connection between algorithmic processing and the user's shopping decision by identifying matching keywords/features, description similarity, category/type compatibility and budget fit.

7.2 FUTURE WORK

7.2.1 Improved Search Ranking

Use richer semantic representations and improved ranking while retaining the DSA engine.

7.2.2 Personalized Recommendations

Use user search and purchase history to personalize ranking.

7.2.3 Real-Time Product Information

Integrate live price, inventory, ratings and availability from authorized sources.

7.2.4 Larger Verified Datasets

Use documented and licensed product datasets with clear provenance.

7.2.5 Advanced Natural-Language Understanding

Improve extraction of product type, features, use case and budget from free-form requirements.

7.2.6 Web or Mobile Interface

Expose the existing search/recommendation engine through a web or mobile front-end.

7.2.7 Automated Benchmarking

Add repeatable performance tests for different catalogue sizes and query classes.

7.2.8 Expanded Test Suite

Add automated tests for every search, recommendation, cart and checkout scenario.

7.3 SKILLS LEARNED

- Problem Analysis and Algorithm Selection
- String Matching and Pattern Processing
- Dynamic Programming and Edit-Distance Calculation
- Suffix Structures and Similarity Processing
- Graph-Based Matching and Maximum Flow
- Optimization and Approximation
- Randomized Algorithms
- Parallel Processing and Java Concurrency
- Java Swing GUI Development
- CSV Parsing and Data Indexing
- Authentication and File-Based Persistence
- Testing and Debugging
- Technical Documentation
- Git/GitHub-oriented Project Organization

REFERENCES

- [1] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, “Fast Pattern Matching in Strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.
- [2] R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.
- [3] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” Soviet Physics Doklady, vol. 10, pp. 707–710, 1966.
- [4] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” SIAM Journal on Computing, vol. 2, no. 4, pp. 225–231, 1973.
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.
- [6] Oracle Java Documentation, Java language, Java Swing, and standard Java libraries.
- [7] DATA STRUCTURES AND ALGORITHMS - 3 (25CS2103E), course handout and project requirements.
- [8] Product Search and Recommendation System for Shopping project source code, data/products.csv, and test evidence.

Table of contents is provided as a fixed report index for easy printing and submission.